

# Lecture 10

## Polymorphism, Part 2 Composition vs. Inheritance

Example Code and Exercises

<https://github.com/eecs498-software-design/lecture>



# Inheritance - Example

**Dependencies:** The derived class depends on the base, but the base also depends on the derived class via overridden methods.

```
abstract class SpellingBeeGame {
    protected pts = 0;

    public submitGuess(word: string): number {
        if (word.length < this.minLength()) {
            console.log(this.getTooShortMsg());
            return 0;
        }

        const pts = this.calcPoints(word);
        this.pts += pts;
        console.log(this.getSuccessMsg(word));
        console.log(`You earned ${pts}!`);
        return pts;
    }

    public getPoints(): number { return this.pts; }

    protected abstract minLength(): number;
    protected calcPoints(word: string): number {
        return word.length; // default points = word length
    }

    protected getTooShortMsg(): string {
        return "Too short!";
    }

    protected getSuccessMsg(word: string): string {
        return `${this.calcPoints(word)} points!`;
    }
}
```

**Control Flow:** It's not hard to end up with both regular and inverted control flow, given the two-way dependency.

```
class HalloweenBee extends SpellingBeeGame {
    private num_too_short = 0;

    override minLength(): number { return 4; }
    override getTooShortMsg() {
        this.num_too_short++;
        return "👻".repeat(this.num_too_short) + " Boo short!";
    }

    override getSuccessMsg(word: string){ return `🎃 "${word}"`; }
}

class SportsBee extends SpellingBeeGame {
    override minLength(): number { return 4; }
    override getTooShortMsg() {return "🏀 Denied!"; }
    override getSuccessMsg(word: string){ return `🏆 "${word}"`; }
}

class EasyBee extends SpellingBeeGame {
    override minLength() { return 3; }
    override calcPoints(word: string) {
        // first word is always worth 10 points
        if (this.pts === 0) { return 10; }
        else { return word.length + 2; } // bonus!
    }
}
```

# Fragile Base Class Problem

```
abstract class SpellingBeeGame {
    protected pts = 0;

    public submitGuess(word: string): number {
        const tooShortMessage = this.getTooShortMsg();
        if (word.length < this.minLength()) {
            console.log(tooShortMessage);
            return this.calcPoints(word);
        }

        this.pts += this.calcPoints(word);
        console.log(this.getSuccessMsg(word) + ` (${pts} points)`);
        console.log(`You earned ${this.calcPoints(word)}!`);
        return this.calcPoints(word);
    }

    public getPoints(): number { return this.pts; }

    protected abstract minLength(): number;
    protected calcPoints(word: string): number {
        return word.length; // default points = word length
    }

    protected getTooShortMsg(): string {
        return "Too short!";
    }

    protected getSuccessMsg(word: string): string {
        return `${this.calcPoints(word)} points!`;
    }
}
```

We make some minor changes to submitGuess() in the base class.

**Bug:** The base class now calls getTooShortMsg() on every guess. That would be OK (even if maybe inefficient), except HalloweenBee added a side effect in that function that increases the number of ghost emojis.

```
class HalloweenBee extends SpellingBeeGame {
    private num_too_short = 0;

    override minLength(): number { return 4; }
    override getTooShortMsg() {
        this.num_too_short++;
        return "👻".repeat(this.num_too_short) + " Boo short!";
    }

    override getSuccessMsg(word: string){ return `🎃 "${word}"`; }
}

class SportsBee extends SpellingBeeGame {
    override minLength(): number { return 4; }
    override getTooShortMsg() {return "🏀 Denied!"; }
    override getSuccessMsg(word: string){ return `🏆 "${word}"`; }
}

class EasyBee extends SpellingBeeGame {
    override minLength() { return 3; }
    override calcPoints(word: string) {
        // first word is always worth 10 points
        if (this.pts === 0) { return 10; }
        else { return word.length + 2; } // bonus!
    }
}
```

**Bug:** The base class now calls this.calcPoints() both before and again after updating this.pts. This causes inconsistent behavior since the EasyBee calcPoints() implementation peeks at this.pts.



# Composition - Example

**Strategy Pattern:** Custom functions for theming and scoring are provided on construction.

```
interface ScoringStrategy {
  minLength(): number;
  calcPoints(word: string, currentPoints: number): number;
}
interface ThemeStrategy {
  getTooShortMsg(): string;
  getSuccessMsg(word: string): string;
}
const StandardScoring: ScoringStrategy = {
  minLength: () => 4,
  calcPoints: (word) => word.length,
};
const EasyScoring: ScoringStrategy = {
  minLength: () => 3,
  calcPoints: (word, currentPoints) => {
    // first word is always worth 10 points
    if (currentPoints === 0) { return 10; }
    else { return word.length + 2; }
  }
};
class HalloweenTheme implements ThemeStrategy {
  private num_too_short = 0;
  getTooShortMsg() {
    this.num_too_short++;
    return "👻".repeat(this.num_too_short) + " Boo short!";
  }
  getSuccessMsg(word: string) { return `🍁 "${word}"`; }
}
const SportsTheme: ThemeStrategy = {
  getTooShortMsg() { return "🏀 Didn't make the cut!"; },
  getSuccessMsg(word: string) { return `🏆 "${word}"`; }
}
```

**Dependencies/Control:**  
One-way from game to strategies.

```
class SpellingBeeGame {
  private pts = 0;

  constructor(private scoring: ScoringStrategy,
               private theme: ThemeStrategy) {}

  public submitGuess(word: string): number {
    const tooShortMessage = this.theme.getTooShortMsg();
    if (word.length < this.scoring.minLength()) {
      console.log(tooShortMessage);
      return this.scoring.calcPoints(word, this.pts);
    }

    const pts = this.scoring.calcPoints(word, this.pts);
    this.pts += pts;
    console.log(this.theme.getSuccessMsg(word));
    console.log(`You earned ${pts}!`);
    return pts;
  }

  public getPoints(): number { return this.pts; }
}

const game1 = new SpellingBeeGame(StandardScoring, new HalloweenTheme());
const game2 = new SpellingBeeGame(EasyScoring, SportsTheme);
const game3 = new SpellingBeeGame(EasyScoring, new HalloweenTheme());
```

**Explicit coupling:** The game+strategies are still behaviorally coupled, but through an explicit interface using params/returns, not implicitly via a shared this context.

# Dependency Injection

- Instead of a class being responsible to **create** its dependencies, they are “**injected**”:
  - Via constructor and stored.
  - To an individual method.

```
class SpellingBeeGame {
  private pts = 0;

  constructor(private scoring: ScoringStrategy,
               private theme: ThemeStrategy) {}

  public submitGuess(word: string): number {
    const tooShortMessage = this.theme.getTooShortMsg();
    if (word.length < this.scoring.minLength()) {
      console.log(tooShortMessage);
      return this.scoring.calcPoints(word, this.pts);
    }

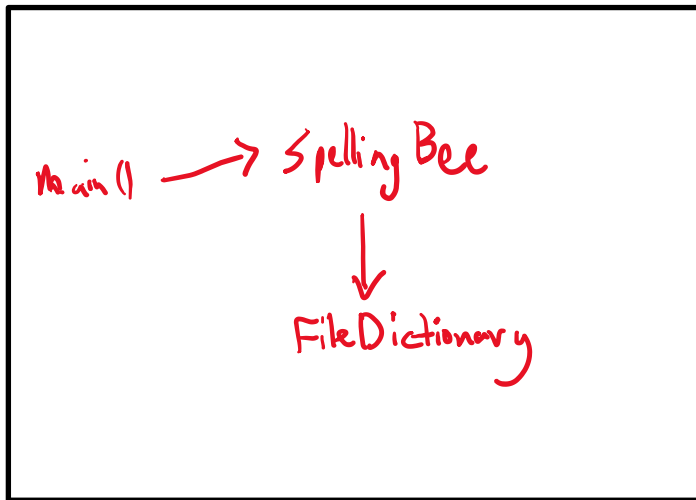
    const pts = this.scoring.calcPoints(word, this.pts);
    this.pts += pts;
    console.log(this.theme.getSuccessMsg(word));
    console.log(`You earned ${pts}!`);
    return pts;
  }

  public getPoints(): number { return this.pts; }
}

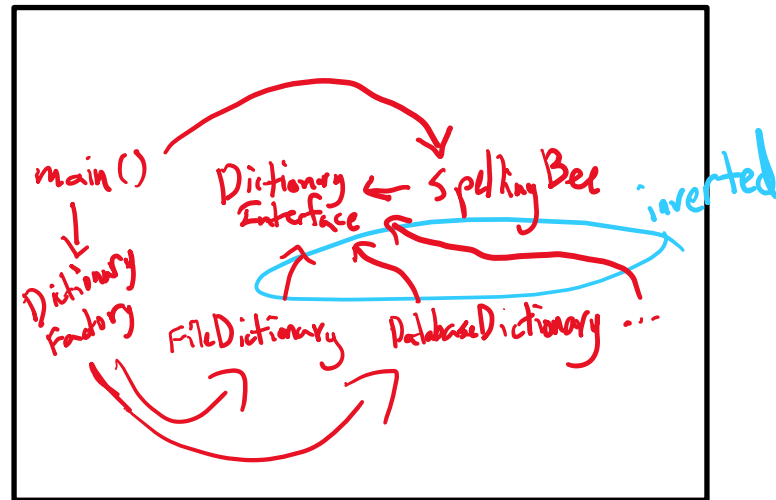
const game1 = new SpellingBeeGame(StandardScoring,
                                  new HalloweenTheme());
const game2 = new SpellingBeeGame(EasyScoring,
                                  SportsTheme());
const game3 = new SpellingBeeGame(EasyScoring,
                                  new HalloweenTheme());
```

# Dependency Inversion

- When abstractions depend on **interfaces** instead of specific, **concrete implementations**, dependencies end up **inverted**.



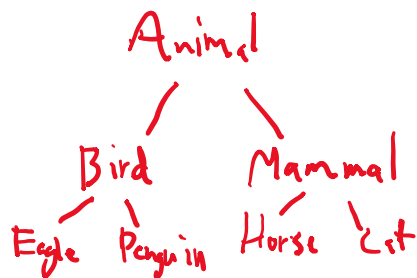
Depend on Specific Implementation



Depend on Interface

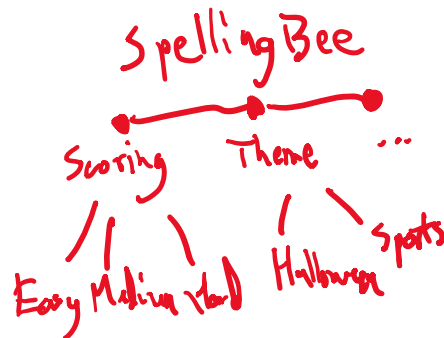
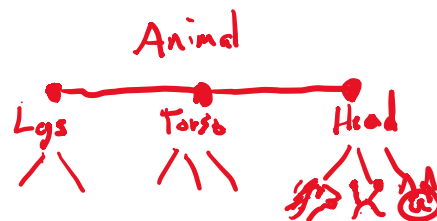
# Polymorphism via Inheritance/Composition

- Inheritance



"real world"  
often doesn't translate  
to code/software

- Composition



# Inheritance Mechanisms

- Inheritance provides several things:
  - **Base Class Subobject**  
*An implicit component used by the base class (derived → base).*
  - **Virtual Functions/Overriding**  
*Inversion of control* since base can call into derived.  
Analogous to *dependency injection* of overrides (base → derived).  
Potential for complex two-way dependency.
  - **protected Access Level**  
Removes encapsulation boundary between base/derived class.





# Break Time

[bit.ly/4rVxFv5](https://bit.ly/4rVxFv5)

